

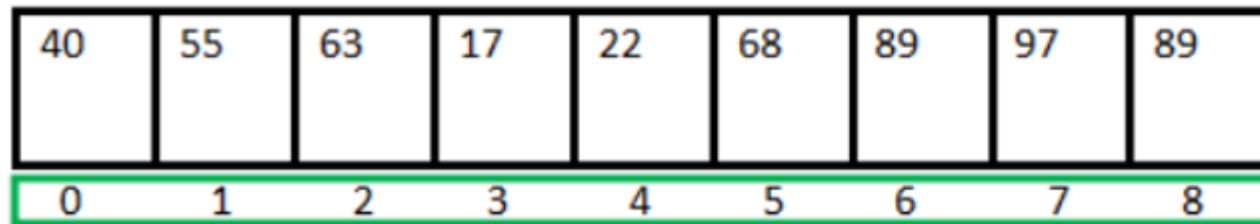
OBJECT ORIENTED PROGRAMMING USING C++



Study Materials

Array:

- Arrays store elements in contiguous memory locations, resulting in easily calculable addresses for the elements stored and this allows faster access to an element at a specific index.



40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

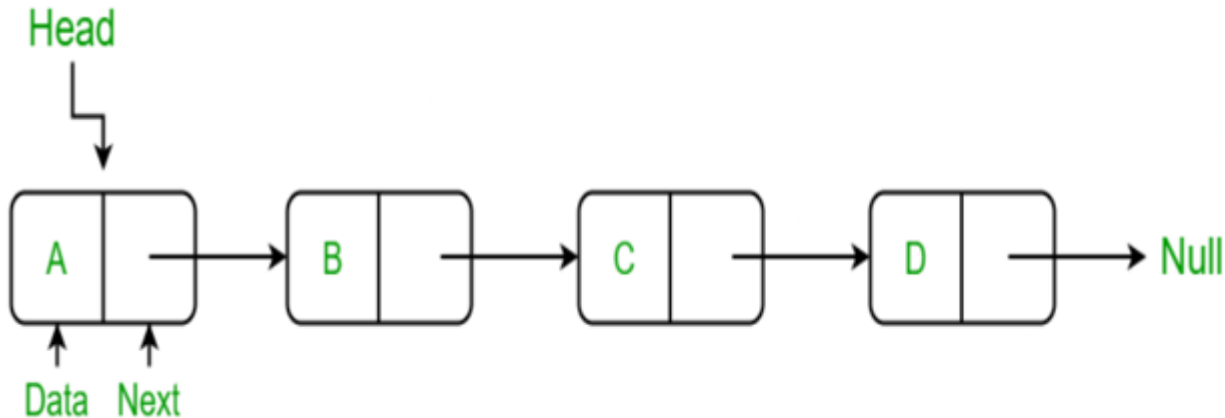
Array Length = 9

First Index = 0

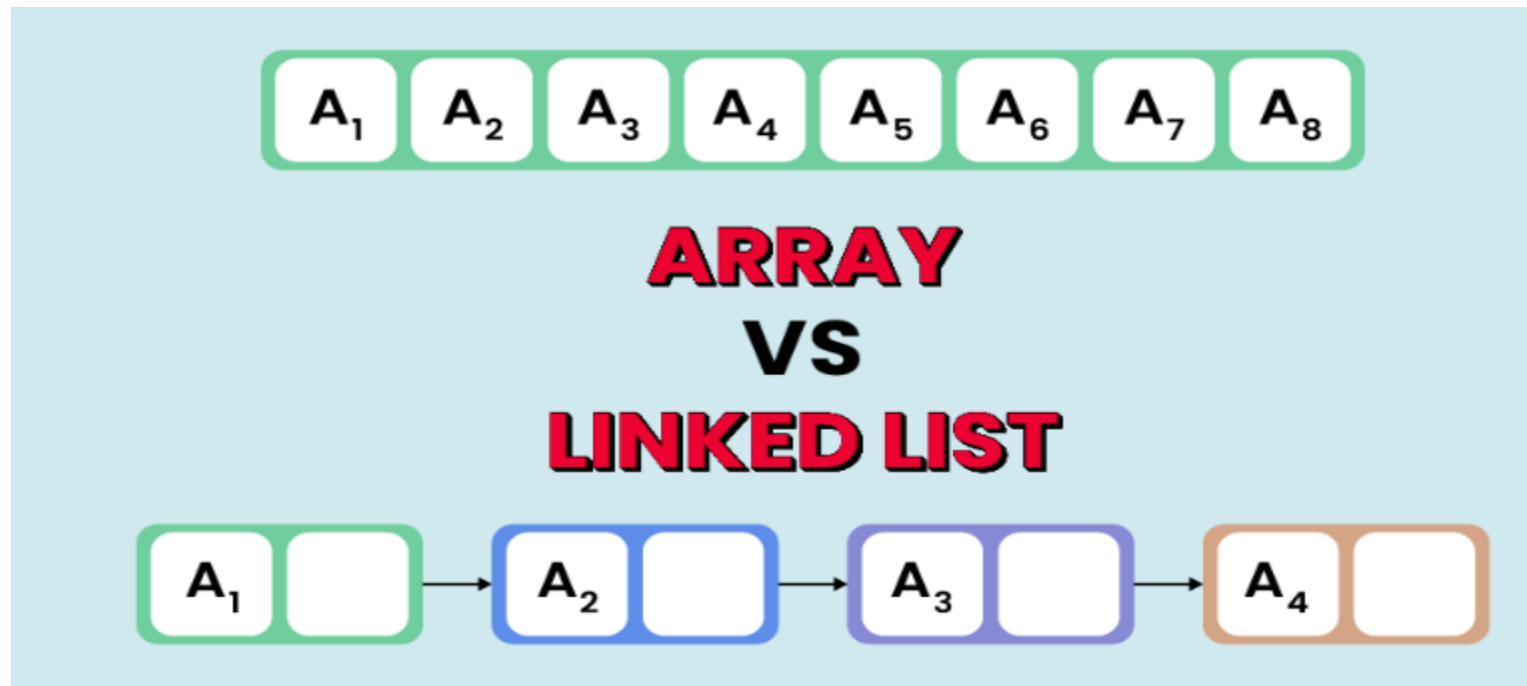
Last Index = 8

Linked List:

- Linked lists are less rigid in their storage structure and elements are usually not stored in contiguous locations, hence they need to be stored with additional tags giving a reference to the next element.



Array vs Linked list:

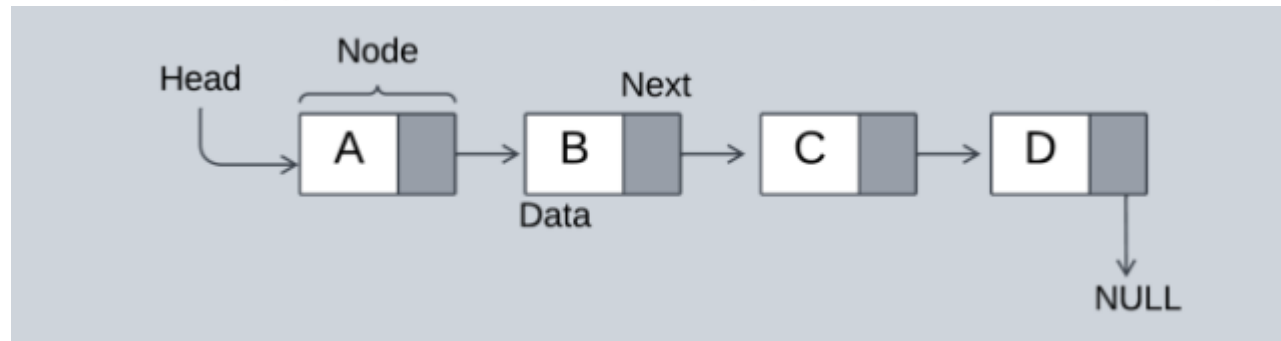


Array vs Linked list:

Aspect	Arrays	Linked Lists
Memory Allocation	Fixed size, contiguous memory block.	Dynamic size, nodes linked by pointers.
Insertion and Deletion	Inefficient, requires shifting elements.	Efficient, only updates node links.
Access	Direct access by index, fast retrieval.	Sequential access, slower traversal.
Memory Usage	Potential wastage if not fully utilised.	Efficient, but nodes use extra memory for pointers.
Flexibility	Fixed size, resizing needs new Array and copy.	Flexible, easily expands or shrinks as needed.

Linked list:

- Linked lists are less rigid in their storage structure and elements are usually not stored in contiguous locations, hence they need to be stored with additional tags giving a reference to the next element.



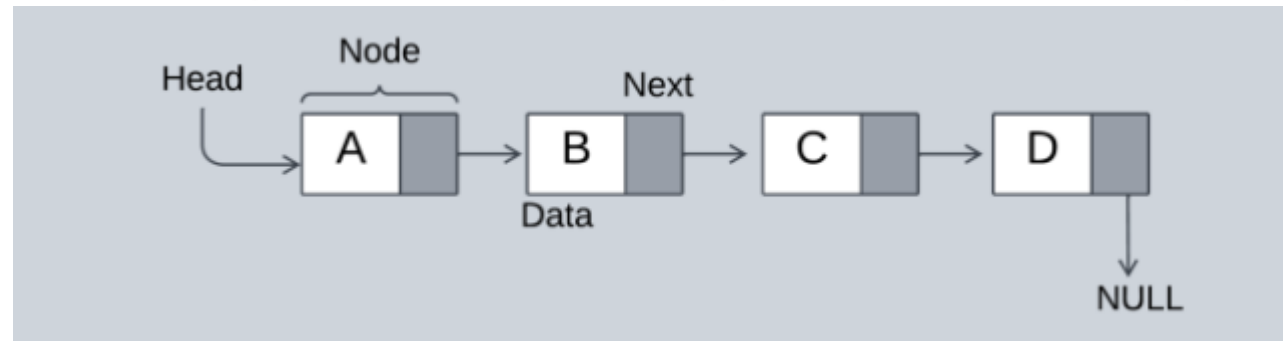
Types of Linked Lists:

Different Types of Linked Lists

- a) Singly Linked List
- b) Doubly Linked List
- c) Circular Linked List
- d) Circular Doubly Linked List

Singly Linked List:

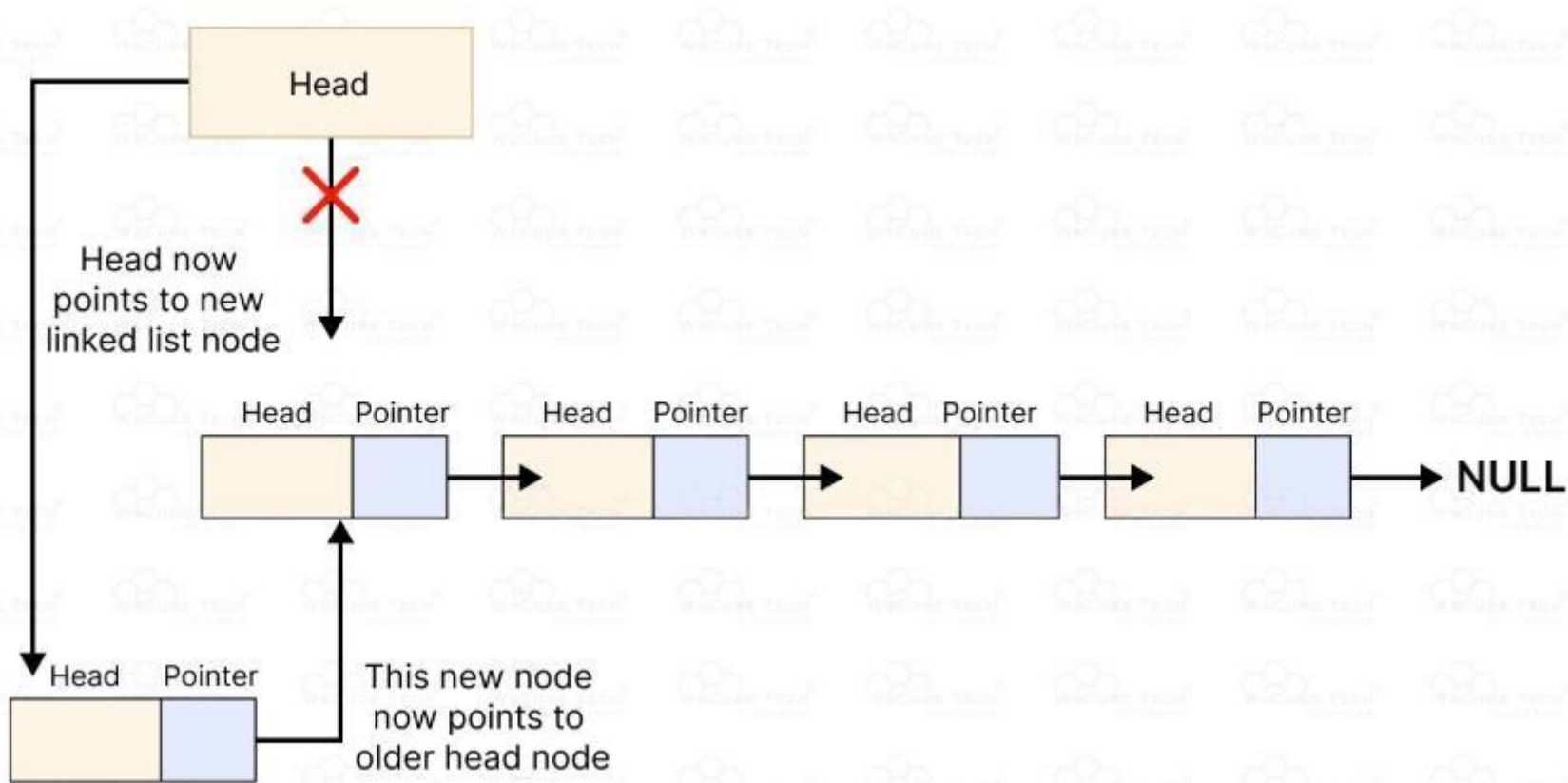
- A singly linked list in data structures is essentially a series of connected elements where each element, known as a node, contains a piece of data and a reference to the next node in the sequence.



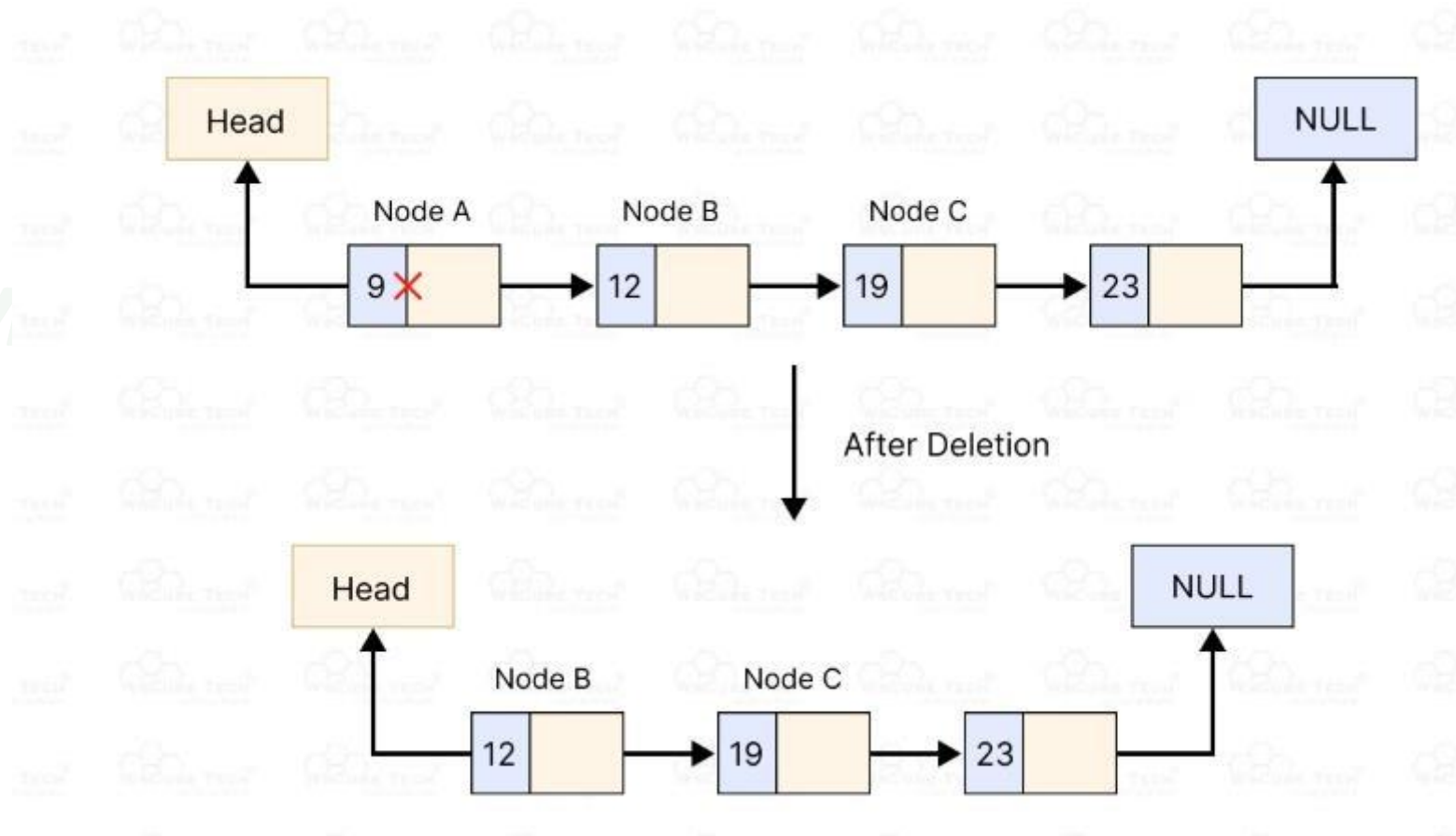
Operations:

- Traversal - access each element of the linked list
- Insertion - adds a new element to the linked list
- Deletion - removes the existing elements
- Search - find a node in the linked list
- Sort - sort the nodes of the linked list

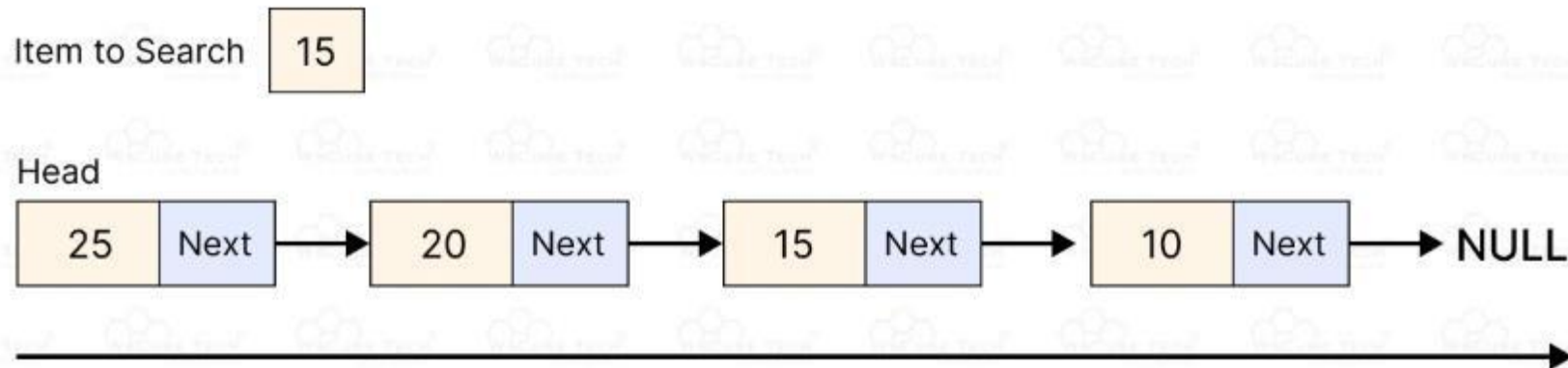
Insertion Operations:



Deletion Operations:



Search Operation:



Linearly Traverse
Checking `node.data == Item`

Advantages:

- Singly linked lists in data structures can grow and shrink during runtime as needed without a predefined size.
- They only allocate memory for nodes that are actually in use, reducing memory waste compared to pre-allocated data structures like arrays.
- Adding or removing nodes doesn't require shifting elements, which can be a costly operation in arrays.
- This is particularly beneficial at the beginning of the list.
- Unlike arrays, linked lists don't reserve memory in advance, which can be more efficient for certain types of applications where the size of the data structure fluctuates.

Disadvantages:

- Accessing an element in a singly linked list requires traversal from the head to the point of interest, which can be time-consuming.
- Each node in a singly linked list requires additional memory for the pointer alongside the actual data.
- Implementing a singly linked list is more complex than using an array, particularly when it comes to handling pointers, which can introduce bugs like memory leaks.